

Battery Energy Storage Optimization

تحسين تشغيل بطارية تخزين الطاقة

State of Charge, Charging, Discharging, Electricity Price, and Optimal Scheduling

حالة الشحن والشحن والتفريغ وأسعار الكهرباء والجدولة المثلى

د. م. أوس غباش | إعداد: Dr.-Ing. Aouss Gabash

Lab Objectives

- Understand battery scheduling as an optimization problem.
- Model the battery State of Charge.
- Include charging and discharging efficiencies.
- Respect SOC and power limits.
- Use electricity price as part of the objective function.
- Compare uncontrolled and optimized battery operation.
- Implement a simple optimization routine in MATLAB.

أهداف المختبر

- فهم جدولة البطارية كمسألة تحسين.
- نمذجة حالة الشحن SOC.
- إضافة كفاءة الشحن والتفريغ.
- احترام حدود SOC والقدرة.
- إدخال سعر الكهرباء ضمن دالة الهدف.
- مقارنة التشغيل غير المحسن بالتشغيل الأمثل.

1. Battery Scheduling Problem

A battery can charge when electricity is cheap and discharge when electricity is expensive.

Low Price $\rightarrow$ Charge

High Price $\rightarrow$ Discharge

The goal is to reduce the total cost of purchasing electricity while respecting battery constraints.

1. مسألة جدولة البطارية

يمكن شحن البطارية عندما يكون سعر الكهرباء منخفضًا وتفريغها عندما يكون السعر مرتفعًا.

الهدف هو تقليل التكلفة الكلية للطاقة مع احترام القيود التشغيلية للبطارية.

2. Time Horizon

Consider a six-hour scheduling horizon:

$$k = 1, 2, \dots, 6$$

Hour	Electricity Price [€/MWh]	Load [MW]
1	50	6.0
2	40	5.5
3	35	5.0
4	70	6.5
5	90	7.0
6	80	6.5

2. أفق الجدولة

نقسم فترة الدراسة إلى ست ساعات، ولكل ساعة سعر للكهرباء وقيمة حمل.

3. Battery Parameters

Parameter	Value
Energy Capacity E _{max}	4 MWh
Minimum SOC	20%
Maximum SOC	90%
Initial SOC	50%
Maximum Charge Power	2 MW
Maximum Discharge Power	2 MW
Charge Efficiency	0.95
Discharge Efficiency	0.95

3. معاملات البطارية

نحدد سعة الطاقة وحدود SOC والاستطاعة القصوى للشحن والتفريغ وكفاءة كل منهما.

4. Battery Power Sign Convention

Define:

$$P_B(k) > 0 \rightarrow \text{Discharging}$$

$$PB(k) < 0 \rightarrow \text{Charging}$$

Therefore:

$$-P_{ch, \max} \leq PB(k) \leq P_{dis, \max}$$

4. إشارة قدرة البطارية

نستخدم الاتفاق التالي:

- موجبة $\rightarrow$ البطارية تفرغ PB
- سالبة $\rightarrow$ البطارية تشحن PB

5. State of Charge

Battery energy is:

$$E(k) = SOC(k)E_{max}$$

and:

$$SOC(k) = E(k)/E_{max}$$

5. حالة الشحن SOC

$$SOC(k) = E(k)/Emax$$

تمثل SOC النسبة بين الطاقة المخزنة حاليًا والسعة الكلية للبطارية.

6. SOC During Charging

If PB is negative, the battery is charging:

$$E(k+1) = E(k) + \eta ch |PB(k)| \Delta t$$

6. أثناء الشحن SOC

$$E(k+1) = E(k) + \eta ch |PB(k)| \Delta t$$

لا تدخل كامل الطاقة القادمة إلى البطارية بسبب وجود خسائر شحن.

7. SOC During Discharging

If PB is positive:

$$E(k+1) = E(k) - PB(k)\Delta t/\eta_{dis}$$

The stored energy decreases slightly more than the delivered electrical energy because efficiency is less than one.

7. SOC أثناء التفريغ

$$E(k+1) = E(k) - PB(k)\Delta t/\eta_{dis}$$

تنخفض الطاقة المخزنة بأكثر قليلاً من الطاقة التي تصل فعلياً إلى الشبكة بسبب خسائر التفريغ.

8. SOC Constraints

$$SOC_{min} \leq SOC(k) \leq SOC_{max}$$

For this battery:

$$0.20 \leq SOC(k) \leq 0.90$$

These limits protect the battery and define the usable energy window.

8. قيود SOC

$$0.20 \leq SOC(k) \leq 0.90$$

يجب ألا تتجاوز حالة الشحن هذه الحدود خلال أي ساعة.

9. Grid Power

Without PV in this first example:

$$P_{grid}(k) = P_{load}(k) - PB(k)$$

If the battery discharges, grid demand decreases.

If the battery charges, grid demand increases.

9. القدرة المسحوبة من الشبكة

$$P_{grid}(k) = P_{load}(k) - PB(k)$$

10. Electricity Cost

For one hour:

$$Cost(k) = Price(k)P_{grid}(k)\Delta t$$

The total operating cost is:

$$J = \sum Price(k)P_{grid}(k)\Delta t$$

10. تكلفة الكهرباء

$$J = \sum Price(k)P_{grid}(k)\Delta t$$

تحاول خوارزمية التحسين اختيار تشغيل البطارية الذي يجعل هذه التكلفة أصغر ما يمكن.

11. Decision Vector

The optimization variables are battery powers over all hours:

$$x = [PB(1), PB(2), \dots, PB(6)]$$

One candidate schedule may be:

$$[-1.5, -1, 0, 0.5, 2, 1]$$

11. متجه القرار

يمثل الحل الكامل قدرة البطارية في كل ساعة:

$$x = [PB(1), PB(2), \dots, PB(6)]$$

12. Why Do We Need Penalties?

A randomly generated schedule may violate SOC limits.

$$SOC(k) < SOC_{min}$$

or:

$$SOC(k) > SOC_{max}$$

Therefore define:

$$J_{pen} = J + Penalty_{SOC}$$

12. لماذا نحتاج إلى العقوبات؟

قد يؤدي جدول تشغيل عشوائي إلى شحن أو تفريغ البطارية خارج الحدود المسموحة.

نضيف عقوبة كبيرة حتى تصبح هذه الحلول غير جذابة للمحسن.

13. SOC Penalty

At every hour:

$$Penalty = \lambda[SOC_{min} - SOC]^2$$

if SOC is too low, and:

$$Penalty = \lambda[SOC - SOC_{max}]^2$$

if SOC is too high.

13. عقوبة SOC

تزداد العقوبة كلما ابتعدت SOC عن المجال المسموح.

14. Final SOC Constraint

Without a final-energy condition, the optimizer may simply empty the battery near the end.

Therefore require:

$$SOC_{final} \approx SOC_{initial}$$

and add:

$$Penalty_{Final} = \lambda f(SOC_{final} - SOC_{initial})^2$$

14. قيد SOC النهائي

إذا لم نفرض شرطًا نهائيًا، فقد تقوم الخوارزمية بتفريغ البطارية بالكامل في نهاية الفترة للحصول على تكلفة منخفضة ظاهريًا.

$$SOC_{final} \approx SOC_{initial}$$

هذا يجعل المقارنة الاقتصادية أكثر عدالة بين بداية ونهاية أفق الجدولة.

15. MATLAB Data

```
price = [50 40 35 70 90 80];  
  
loadMW = [6.0 5.5 5.0 6.5 7.0 6.5];  
  
N = length(price);  
  
dt = 1;
```

15. البيانات في MATLAB

نخزن أسعار الكهرباء والحمل لكل ساعة، ونفترض أن طول الفترة الزمنية ساعة واحدة.

16. Battery Parameters in MATLAB

```
Emax = 4;  
  
SOCmin = 0.20;  
SOCmax = 0.90;  
SOC0 = 0.50;  
  
PchMax = 2;  
PdisMax = 2;  
  
etaCh = 0.95;  
etaDis = 0.95;
```

16. معاملات البطارية في MATLAB

نعرف سعة البطارية وحدود SOC والقدرة القصوى وكفاءات التشغيل.

17. Evaluate One Schedule

Start with:

$$E(1) = SOC0Emax$$

Then simulate every hour sequentially.

```

E = SOC0*Emax;

for k = 1:N

    PB = schedule(k);

    if PB < 0

        E = E + etaCh*(-PB)*dt;

    else

        E = E - PB*dt/etaDis;

    end

    SOC = E/Emax;

end

```

17. تقييم جدول تشغيل واحد

يجب حساب SOC ساعة بعد ساعة، لأن حالة البطارية الحالية تعتمد على جميع قرارات الشحن والتفريغ السابقة.

18. Cost Calculation

```

Pgrid = loadMW(k) - PB;

cost = cost + price(k)*Pgrid*dt;

```

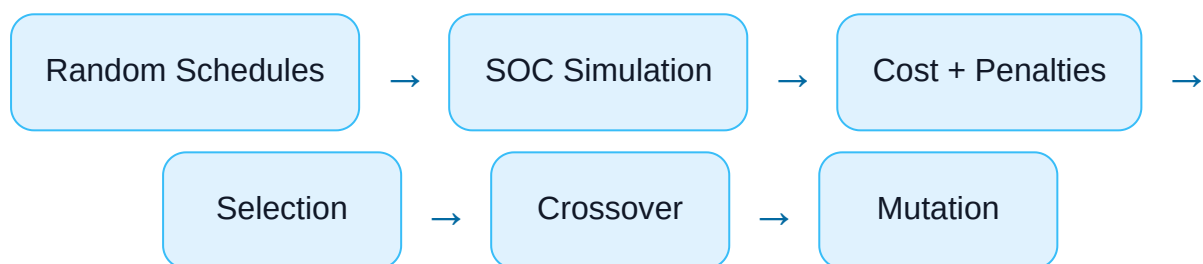
The cost is accumulated across the scheduling horizon.

18. حساب التكلفة

نحسب في كل ساعة القدرة المسحوبة من الشبكة، ثم نضربها بالسعر الزمني.

19. Evolutionary Optimizer

We use a simple population-based optimizer.



19. خوارزمية التحسين

يحتوي كل فرد في المجتمع على جدول كامل لقدرة البطارية عبر الساعات الست.

20. Complete MATLAB Program


```

clear; clc; close all;

% -----
% Battery Energy Storage Optimization
% Educational evolutionary scheduling example
% -----

price = [50 40 35 70 90 80];

loadMW = [6.0 5.5 5.0 6.5 7.0 6.5];

N = length(price);

dt = 1;

% -----
% Battery parameters
% -----

Emax = 4;

SOCmin = 0.20;
SOCmax = 0.90;

SOC0 = 0.50;

PchMax = 2;
PdisMax = 2;

etaCh = 0.95;
etaDis = 0.95;

% -----
% Optimization parameters
% -----

popSize = 60;
maxGen = 150;

```

```

Pc = 0.8;
Pm = 0.12;

sigmaMut = 0.4;

lambdaSOC    = 1e5;
lambdaFinal  = 1e5;

% -----
% Initial population
% Each row is one battery schedule
% -----

population = ...
    -PchMax + ...
    rand(popSize,N)*(PchMax+PdisMax);

bestHist = zeros(maxGen,1);

% -----
% Evolutionary loop
% -----

for gen = 1:maxGen

    Jvec = zeros(popSize,1);

    for i = 1:popSize

        schedule = population(i,:);

        E = SOC0*Emax;

        totalCost = 0;
        penaltySOC = 0;

        for k = 1:N

            PB = schedule(k);

```

```

% -----
% Battery energy update
% -----

if PB < 0

    E = E + ...
        etaCh*(-PB)*dt;

else

    E = E - ...
        PB*dt/etaDis;

end

SOC = E/Emax;

% -----
% SOC penalties
% -----

if SOC < SOCmin

    penaltySOC = penaltySOC + ...
        lambdaSOC*(SOCmin-SOC)^2;

elseif SOC > SOCmax

    penaltySOC = penaltySOC + ...
        lambdaSOC*(SOC-SOCmax)^2;

end

% -----
% Grid power and electricity cost
% -----

Pgrid = loadMW(k) - PB;

```

```

        totalCost = ...
            totalCost + ...
            price(k)*Pgrid*dt;

    end

    % -----
    % Final SOC penalty
    % -----

    SOCfinal = E/Emax;

    penaltyFinal = ...
        lambdaFinal*(SOCfinal-SOC0)^2;

    % -----
    % Total objective
    % -----

    Jvec(i) = ...
        totalCost + ...
        penaltySOC + ...
        penaltyFinal;

end

% -----
% Elitism
% -----

[bestJ,bestIdx] = min(Jvec);

elite = population(bestIdx,:);

bestHist(gen) = bestJ;

% -----
% New population
% -----

```

```

newPopulation = zeros(size(population));

newPopulation(1,:) = elite;

row = 2;

while row <= popSize

    % -----
    % Tournament selection - parent 1
    % -----

    a = randi(popSize);
    b = randi(popSize);

    if Jvec(a) < Jvec(b)
        parent1 = population(a,:);
    else
        parent1 = population(b,:);
    end

    % -----
    % Tournament selection - parent 2
    % -----

    a = randi(popSize);
    b = randi(popSize);

    if Jvec(a) < Jvec(b)
        parent2 = population(a,:);
    else
        parent2 = population(b,:);
    end

    % -----
    % Arithmetic crossover
    % -----

    if rand < Pc

```

```

        alpha = rand;

        child1 = ...
            alpha*parent1 + ...
            (1-alpha)*parent2;

        child2 = ...
            (1-alpha)*parent1 + ...
            alpha*parent2;

    else

        child1 = parent1;
        child2 = parent2;

    end

% -----
% Mutation
% -----

for k = 1:N

    if rand < Pm

        child1(k) = ...
            child1(k) + ...
            sigmaMut*randn;

    end

    if rand < Pm

        child2(k) = ...
            child2(k) + ...
            sigmaMut*randn;

    end

end

end

```

```

% -----
% Battery power limits
% -----

child1 = ...
    min(max(child1,-PchMax),PdisMax);

child2 = ...
    min(max(child2,-PchMax),PdisMax);

% -----
% Store offspring
% -----

newPopulation(row,:) = child1;

if row+1 <= popSize
    newPopulation(row+1,:) = child2;
end

row = row + 2;

end

population = newPopulation;

end

% -----
% Final evaluation
% -----

Jvec = zeros(popSize,1);

for i = 1:popSize

    schedule = population(i,:);

    E = SOC0*Emax;

```

```

totalCost = 0;
penaltySOC = 0;

for k = 1:N

    PB = schedule(k);

    if PB < 0

        E = E + ...
            etaCh*(-PB)*dt;

    else

        E = E - ...
            PB*dt/etaDis;

    end

    SOC = E/Emax;

    if SOC < SOCmin

        penaltySOC = penaltySOC + ...
            lambdaSOC*(SOCmin-SOC)^2;

    elseif SOC > SOCmax

        penaltySOC = penaltySOC + ...
            lambdaSOC*(SOC-SOCmax)^2;

    end

    Pgrid = loadMW(k) - PB;

    totalCost = ...
        totalCost + ...
        price(k)*Pgrid*dt;

```



```

end

SOCfinal = E/Emax;

penaltyFinal = ...
    lambdaFinal*(SOCfinal-SOC0)^2;

Jvec(i) = ...
    totalCost + ...
    penaltySOC + ...
    penaltyFinal;

end

[bestJ,bestIdx] = min(Jvec);

bestSchedule = population(bestIdx,:);

% -----
% Simulate best schedule
% -----

SOCbest = zeros(1,N+1);
PgridBest = zeros(1,N);

SOCbest(1) = SOC0;

E = SOC0*Emax;

bestCost = 0;

for k = 1:N

    PB = bestSchedule(k);

    if PB < 0

        E = E + ...
            etaCh*(-PB)*dt;

```

```

else

    E = E - ...
        PB*dt/etaDis;

end

SOCbest(k+1) = E/Emax;

PgridBest(k) = ...
    loadMW(k) - PB;

bestCost = ...
    bestCost + ...
    price(k)*PgridBest(k)*dt;

end

% -----
% Cost without battery
% -----

baseCost = ...
    sum(price .* loadMW * dt);

saving = baseCost - bestCost;

% -----
% Display results
% -----

fprintf('Battery Optimization Result\n\n')

fprintf('Base cost without battery = %.2f\n', ...
    baseCost)

fprintf('Optimized cost = %.2f\n', ...
    bestCost)

fprintf('Cost saving = %.2f\n\n', ...

```

```

        saving)

fprintf('Battery schedule [MW]:\n')
disp(bestSchedule)

fprintf('SOC trajectory:\n')
disp(SOCbest)

% -----
% Plot convergence
% -----

figure

plot(1:maxGen,bestHist,'LineWidth',1.6)

grid on

xlabel('Generation')
ylabel('Best Objective')

title('Battery Scheduling Optimization Convergence')

% -----
% Plot battery power
% -----

figure

bar(1:N,bestSchedule)

grid on

xlabel('Hour')
ylabel('Battery Power [MW]')

title('Optimal Battery Schedule')

% -----
% Plot SOC

```

```

% -----

figure

plot(0:N,SOCbest,'o-','LineWidth',1.6)

hold on

yline(SOCmin,'--')
yline(SOCmax,'--')

grid on

xlabel('Hour')
ylabel('State of Charge')

title('Battery SOC Trajectory')

% -----
% Plot electricity price
% -----

figure

stairs(1:N,price,'LineWidth',1.6)

grid on

xlabel('Hour')
ylabel('Electricity Price [EUR/MWh]')

title('Electricity Price Profile')

```

20. البرنامج الكامل في MATLAB

ينشئ البرنامج عدة جداول تشغيل مرشحة، ثم يحاكي SOC لكل جدول، ويحسب تكلفة شراء الطاقة والعقوبات، ثم يطور الحلول عبر الأجيال.

النتيجة المتوقعة منطقياً: تميل البطارية إلى الشحن في الساعات الرخيصة والتفريغ في الساعات الأعلى سعراً، ضمن حدودها التشغيلية.

21. Cost Without the Battery

The reference operating cost is:

$$J_{base} = \sum Price(k) Load(k) \Delta t$$

MATLAB:

```
baseCost = sum(price .* loadMW * dt);
```

21. التكلفة دون استخدام البطارية

نحسب أولاً ما كانت ستدفعه المنشأة لو اشترت كامل الطاقة من الشبكة دون بطارية.

22. Economic Saving

$$Saving = J_{base} - J_{optimized}$$

A positive value means the optimized battery schedule reduced electricity purchasing cost.

$$Saving = J_{base} - J_{optimized}$$

إذا كانت القيمة موجبة فهذا يعني أن البطارية ساهمت في تقليل تكلفة الطاقة.

23. Interpreting the Optimal Schedule

Negative PB

Battery charging.

Positive PB

Battery discharging.

Low Price Hours

Charging is economically attractive.

High Price Hours

Discharging may reduce expensive grid purchases.

23. تفسير جدول التشغيل الأمثل

راقب علاقة سعر الكهرباء بإشارة قدرة البطارية.

إذا كانت الخوارزمية تعمل بصورة جيدة، يجب أن يظهر منطق اقتصادي واضح في النتائج، وليس مجرد أرقام عشوائية.

24. Experiment 1 — Change Initial SOC

$SOC0 = 0.30;$

$SOC0 = 0.50;$

$SOC0 = 0.80;$

Compare optimal schedules and savings.

24. التجربة 1 — تغيير SOC الابتدائية

تؤثر الطاقة المتاحة عند بداية الجدولة على قدرة البطارية على الاستفادة من الأسعار المرتفعة لاحقاً.

25. Experiment 2 — Change Battery Capacity

$E_{max} = 2;$

$E_{max} = 4;$

$E_{max} = 8;$

A larger battery can shift more energy, but larger capacity does not automatically mean proportional economic benefit.

25. التجربة 2 — تغيير سعة البطارية

قارن الوفرة الاقتصادي عند استخدام بطاريات بأحجام مختلفة.

26. Experiment 3 — Change Efficiency

```
etaCh = 1.00;  
etaDis = 1.00;
```

Then compare with:

```
etaCh = 0.90;  
etaDis = 0.90;
```

Lower efficiency reduces the economic value of energy arbitrage.

26. التجربة 3 — تغيير الكفاءة

كلما انخفضت كفاءة الشحن والتفريغ، ازدادت خسائر الطاقة وانخفضت فائدة البطارية الاقتصادية.

27. Experiment 4 — Flat Electricity Price

Try:


```
price = [60 60 60 60 60 60];
```

Ask whether battery arbitrage still provides an economic benefit.

27. التجربة 4 — سعر كهرباء ثابت

إذا كان السعر نفسه في جميع الساعات، تختفي تقريبًا فرصة شراء الطاقة رخيصة وبيعها أو استخدامها عند سعر أعلى.

هذا يوضح أن قيمة التخزين تعتمد على طبيعة النظام والسوق، وليس على البطارية وحدها.

28. Add PV Generation

A natural extension is:

$$P_{grid}(k) = P_{load}(k) - PPV(k) - PB(k)$$

The battery can then store excess PV energy and use it later.

28. إضافة توليد PV

$$P_{grid}(k) = P_{load}(k) - PPV(k) - PB(k)$$

29. Battery Degradation Cost

Real optimization should also consider battery aging.

A simple educational approximation is:

$$C_{deg} = c_{deg} \sum |PB(k)| \Delta t$$

Then:

$$J = EnergyCost + DegradationCost$$

Without degradation cost, the optimizer may cycle the battery more aggressively than is economically realistic.

29. تكلفة تدهور البطارية

كل عملية شحن وتفريغ تساهم في استهلاك عمر البطارية.

$$C_{deg} = c_{deg} \sum |PB(k)| \Delta t$$

30. From Scheduling to Smart Energy Management

A more advanced energy-management problem may optimize:

$$x = [BatteryPower, PVCurtailment, FlexibleLoad, GridExchange,$$

and minimize:

$$J = EnergyCost + Losses + BatteryDegradation + Penalties$$

30. من جدولة البطارية إلى إدارة الطاقة الذكية

يمكن توسيع المسألة لاحقًا لتشمل PV والأحمال المرنة والتبادل مع الشبكة والفواقد وتدهور البطارية.

31. Lab Tasks

1. Enter the electricity-price and load profiles.
2. Implement charging and discharging equations.
3. Calculate SOC hour by hour.
4. Add SOC penalties.
5. Add the final-SOC condition.
6. Run the evolutionary optimizer.

7. Plot the optimal battery schedule.
8. Plot the SOC trajectory.
9. Calculate the cost without a battery.
10. Calculate the optimized cost and savings.
11. Change the initial SOC.
12. Change battery capacity and efficiency.
13. Test a flat electricity-price profile.
14. Explain how PV could be added to the model.

31. مهام المختبر

1. أدخل أسعار الكهرباء والحمل.
2. نفذ معادلات الشحن والتفريغ.
3. احسب SOC ساعة بعد ساعة.
4. أضف عقوبات SOC.
5. أضف شرط SOC النهائي.
6. شغل خوارزمية التحسين.
7. ارسم جدول قدرة البطارية الأمثل.
8. ارسم مسار SOC.
9. احسب التكلفة دون البطارية.
10. احسب التكلفة المحسنة والوفر.
11. غيّر SOC الابتدائية.
12. غيّر سعة البطارية وكفاءتها.
13. اختبر حالة السعر الثابت.
14. اشرح كيف يمكن إضافة نظام PV.



Review Questions

1. What does positive battery power mean in this laboratory?
2. How does charging efficiency affect SOC?
3. Why is SOC calculated sequentially?

4. Why do we need SOC limits?
5. Why is a final-SOC constraint useful?
6. Why does the battery tend to charge during low-price hours?
7. How does low efficiency affect economic savings?
8. Why may a flat electricity price reduce the value of storage?
9. Why should degradation cost be included in more realistic studies?
10. How would PV generation modify the grid-power equation?

أسئلة المراجعة

1. ماذا تعني قدرة بطارية موجبة في هذا المختبر؟
2. كيف تؤثر كفاءة الشحن على SOC؟
3. لماذا نحسب SOC بصورة متسلسلة زمنيًا؟
4. لماذا نحتاج إلى حدود SOC؟
5. لماذا نحتاج إلى شرط SOC النهائي؟
6. لماذا تميل البطارية إلى الشحن عند انخفاض السعر؟
7. كيف تؤثر الكفاءة المنخفضة على الوفرة الاقتصادي؟
8. لماذا يقل دور البطارية عند ثبات سعر الكهرباء؟
9. لماذا نضيف تكلفة تدهور البطارية في الدراسات الواقعية؟
10. كيف يتغير قانون قدرة الشبكة عند إضافة PV؟